Laboratorio 6 - Evaluación, toma de decisiones y ajuste de modelos

En este laboratorio vamos a terminar de trabajar con el dataset de SCADA de la turbina aerogenerador.

Para ello, vamos a dividir el laboratorio en dos partes.

1. Evaluación del modelo para la toma de decisiones y definición de umbrales de clasificación

   Continuaremos el proceso que hemos estado siguiendo en los anteriores para clasificar cada uno de los 5 errores presentes en nuestro dataset.

   En este sentido, estudiaremos como los umbrales de clasificación pueden ayudarnos a optimizar el rendimiento y favorecer que no se incurran en determinados fallos. Esto es especialmente relevante para definir la forma en la que se toma esta decisión.

   A su vez, usaremos la matriz de confusión para medir el equilibrio y la precisión de nuestras predicciones de clasificación, así como visualizar el porcentaje total de veces que se pronostica cada clase.

2. Ajuste del modelo para detección multiclase

   En la segunda parte del laboratorio vamos a retomar los datos iniciales sin el procesamiento que hemos estado realizando hasta ahora y sin considerar las ventanas temporales definidas (de 24, 28 y 72h). Por lo tanto, ahora vamos a crear un modelo capaz de detectar la aparición de algún error en un instante determinado. Al ser un problema mucho más complejo, vamos a estudiar el ajuste de parámetros y las técnicas de aumento de datos para mejorar el rendimiento del problema

1.1 Evaluación del modelo para la toma de decisiones

Como se ha introducido, esta primera parte es continuista con el trabajo de los laboratorios previos y, por tanto, vamos a emplear las predicciones y probabilidades obtenidas en ellos.

1. Comenzamos importando los archivos necesarios, en este caso, las predicciones realizadas y sus valores de probabilidad y los datos de entrada X y salida Y.

   En este caso, para tener un ejemplo que permita comprender mejor el comportamiento de los diferentes umbrales de clasificación, se han modificado los parámetros de entrenamiento del laboratorio anterior (laboratorio 5) de forma que se ha conseguido, de manera consciente, obtener resultados más variados y un umbral mayor de probabilidad.

   A este respecto, en este caso los resultados han sido obtenidos con el clasificador Gradien Boosting y entrenando modelos con un menor tamaño de manera que se evite el overfitting.

```
LOCALFILE_CLASS =
'/data/class_predictions2.csv'  dataframe_class_predictions     =
pd.read_csv(LOCALFILE_CLASS)
dataframe_class_predictions['DateTime']                          =
pd.to_datetime(dataframe_class_predictions['DateTime'])
dataframe_class_predictions      =
dataframe_class_predictions.sort_values('DateTime')
```

2. Vamos a continuar trabajando con la ventana de 72h. Por ello, extraemos las variables de outcomes de interés

```
YVarsList   = list(dataframe_outcomes.columns)[1:]

YVars24 = [a for a in YVarsList if '_24' in a]
YVars48 = [a for a in YVarsList if '_48' in a]
YVars72 = [a for a in YVarsList if '_72' in a]

#Assign YVars to 72-hr window
YVars = YVars72
```

3. Ahora vamos a filtrarlo a un determinado rango temporal, en este caso entre el 1/06/2014 y el 20/1/2013

```
dataframe_prob_predictions2 = deepcopy(dataframe_prob_predictions)

XVar  =  'DateTime'  dt1   =
datetime(2014,6,1)  dt2   =
datetime(2015,1,20)

DTs   = list(dataframe_prob_predictions2['DateTime'])
MinI = bisect_left(DTs,dt1) MaxI =
bisect_right(DTs,dt2)
dataframe_prob_predictions2 =
dataframe_prob_predictions2.iloc[MinI:MaxI-1]
```

4. Definimos un método de apoyo para producir fallos en un intervalo de tiempo determinado. Este código devuelve el subconjunto de fallos dentro de ese intervalo.

```
    def GetFaults(MinDT,MaxDT,Fault):
        DTs = list(dataframe_faults['DateTime'])
        MinI = bisect_left(DTs,MinDT)
    MaxI = bisect_right(DTs,MaxDT)
        dataframe_faults2 = dataframe_faults.iloc[MinI:MaxI-1]
    if len(dataframe_faults2)<1: return []
        dataframe_faults2=dataframe_faults2[dataframe_faults2['Fault']==
    Fault]
        if len(dataframe_faults2)<1: return []
return list(dataframe_faults2['DateTime'])
```

5. Ahora vamos a comparar visualmente los datos de los resultados reales con los de las probabilidades obtenidas para cada clasificación

```
 for YVar in
YVars:
    print('DateTime Range: ',dt1,'to',dt2)
sns.set_style("darkgrid")      plt.figure(figsize=(10,
4))


    faults = GetFaults(dt1,dt2,YVar[:2])
YVals = [0.0,1.0]      for dt in faults:
XVals = [dt,dt]
        plt.plot(XVals,YVals,color='r')


    XVals = dataframe_prob_predictions2[XVar]
    YVals = dataframe_prob_predictions2[YVar]


    plt.plot(XVals,YVals)


    adf = plt.gca().get_xaxis().get_major_formatter()
adf.scaled[1.0] = '%m-%d'      plt.ylabel(YVar)
plt.xlabel(XVar)      plt.ylim([-0.05,1.05])
plt.show()
```

Este código compara el rendimiento predictivo de nuestras predicciones a 72 horas con los datos reales agregados de la variable de resultado de la misma ventana. Podemos interpretar el rendimiento del modelo identificando picos en nuestras predicciones (la línea azul) frente a los fallos reales (las líneas rojas).

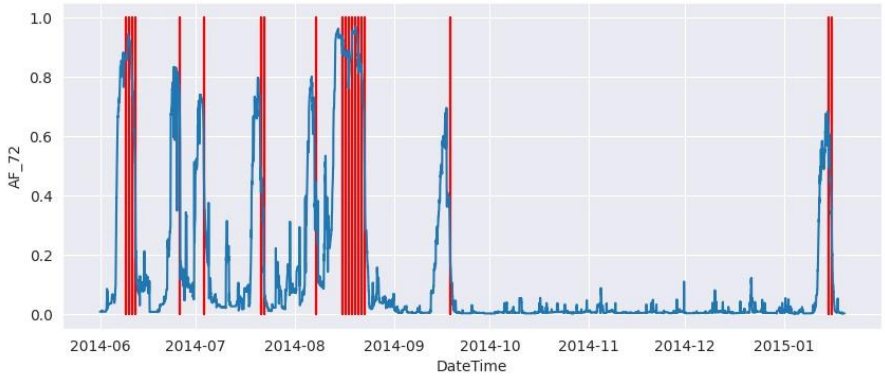


Figura 1: Representación de los fallos reales y su probabilidad en la clasificación para AF

Estas gráficas comparan el rendimiento predictivo de nuestras predicciones a 72 horas con los datos reales agregados de la variable de resultado fallos de la misma ventana.

Podemos interpretar el rendimiento del modelo identificando picos en nuestras predicciones (la línea azul) frente a los fallos reales (las líneas rojas).

Podemos observar cómo se detectan errores para cada pico de la gráfica, lo que nos indica que, con la subida de la probabilidad a partir de determinado umbral, estas probabilidades se convierten en clasificaciones de errores.
Sin embargo, aunque este tipo de representaciones resultan útiles para ver de manera visual cómo se comportan los modelos, no nos dan una percepción cuantitativa de como de acertados

son en sus predicciones. A este respecto, en cambio, las matrices de confusión nos ofrecen una representación más nutrida de cómo se comporta realmente un modelo

Matriz de confusión

Cuando se intenta clasificar (predecir) una variable booleana, puede haber 4 resultados diferentes para una predicción.

En este caso, estamos prediciendo la existencia de un suceso (un fallo). En consecuencia, estamos prediciendo si se produce o no un suceso para una observación determinada.

Por lo tanto, podemos ser "correctos" de 2 formas e "incorrectos" de 2 formas.

Formas en que el modelo puede ser correcto:

- Predecir correctamente que ocurre un suceso: "Verdadero positivo" (TP)
- Predecir correctamente que no ocurre ningún suceso: "Verdadero Negativo"

(TN) Formas en que el modelo puede equivocarse:

- Predecir incorrectamente un suceso: "Falso positivo (FP)
- Predecir incorrectamente que no se produce ningún suceso: "Falso negativo (FN)

Estos resultados pueden organizarse en una matriz para analizar el rendimiento de un clasificador ML. Esto se denomina matriz de confusión.

A continuación, vamos a representar la matriz de confusión normalizada de dos maneras:

- Normalización con sumatorio total al 100%

La primera de las formas nos va a indicar el impacto de cada tipo de predicción en relación con el número total de predicciones. De esta forma, queremos ver el porcentaje de veces que, en el periodo de tiempo seleccionado, se daría cada circunstancia.

Este método implica dividir cada valor de la matriz de confusión por el número total de observaciones. Refleja la proporción de cada tipo de predicción (verdaderos positivos, falsos positivos, verdaderos negativos, falsos negativos) en relación con el conjunto total de predicciones.

Por lo tanto, esta matriz de confusión está normalizada de modo que todas las entradas están en porcentajes del total de predicciones. Esto significa que el sumatorio de todos los valores en la matriz será 100%.

Esto por tanto nos da información del riesgo que tomamos con el diseño de nuestro modelo, ya que nos permite, por ejemplo, identificar el porcentaje de veces que un fallo real podría pasar desapercibido.

- Normalización por filas (Matriz de confusión común)

Esta matriz de confusión es útil para comprender cómo de bien se está clasificando cada clase. De esta forma, Las celdas muestran el número de predicciones para cada categoría como un porcentaje de la cantidad real de esa clase, lo que indica qué porcentaje de la clase real fue correctamente clasificado o mal clasificado.

En este caso, esta matriz de confusión muestra el rendimiento del clasificador para cada clase por separado. En un escenario de clasificación desequilibrado, donde una de las clases es mucho más frecuente, la normalización por filas puede ser especialmente útil para evaluar la capacidad del modelo para identificar la clase menos frecuente.

6. Vamos a comenzar definiendo un método para ver la distribución de producciones entre clases, es decir, normalización con sumatorio total al 100%

   `[Ver código en colab]`

7. Ahora vamos a representar una matriz de confusión para los resultados de predicción de cada variable de resultado. Lo que queremos ver con esto es el porcentaje de veces que nuestro modelo sería capaz de reconocer de manera errónea o correcta cada situación. Esta está basada en el porcentaje de resultados que entran en las cuatro categorías ilustradas anteriormente (verdadero negativo/positivo, falso negativo/positivo)

   `[Ver código en colab]`

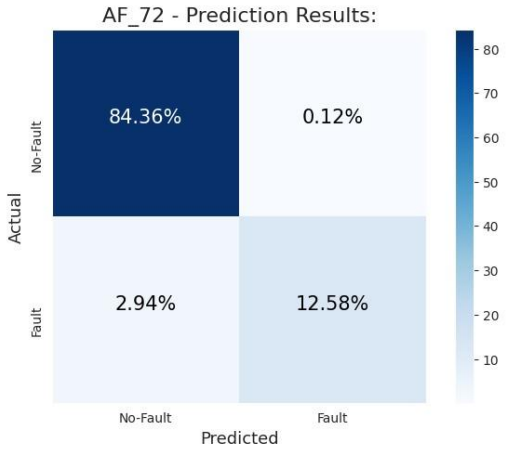


Figura 2: Matriz de confusión normalizada con sumatorio total 100%

Si comparamos la salida de la matriz de confusión para los 5 fallos, podemos observar que el modelo se comporta de una manera bastante similar para cada fallo en valores absolutos.

También podemos observar cómo, fruto del desbalance en los datos, el porcentaje mayor en número de clasificación para cada caso es corresponde a no fallos que, efectivamente, no correspondían a fallos (True Negative). Este desbalance también se manifiesta en que los fallos verdaderos (True Positive) representan un porcentaje muy

inferior. En el caso de AF, por ejemplo, del 100% de los datos clasificados, únicamente el 12,58% de las veces existía (y se detectó) de manera correcta un fallo.

A su vez, si nos fijamos en el porcentaje de muestras reconocidas incorrectamente, podemos observar que el 0,12 y el 2,94% de las veces hubo un resultado de falso negativo o falso positivo. En este caso, esto nos demuestra que en un aproximadamente un 3% del total de las ocasiones, existiendo un fallo, esto no fue detectado. Por el contrario, el modelo demostró ser muy bueno alertando de los fallos cuando, efectivamente, estos han ocurrido, ya que sólo en un 0,12% de las veces hay un error por detectar un falso positivo.

8. A continuación, vamos a representar la matriz de confusión normalizada por filas para ver realmente el rendimiento del clasificador en cada caso

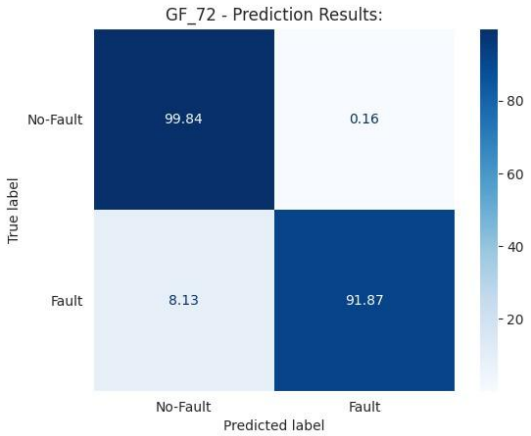


Figura 3: Matriz de confusión normalizada por filas

En este caso podemos observar como el porcentaje de acierto tanto para los verdaderos negativos (TF) como en los verdaderos positivos es muy alto. A su vez, los casos en los que el modelo tuvo errores es muy bajo. Esto, aunque representado de una manera diferente, se corresponde con la información obtenida en la matriz de confusión previa.

De esta forma, nuevamente en el ejemplo de AF, podemos ver que el 99.84% de las veces que no había ningún fallo, el modelo lo predijo correctamente. Por lo tanto, el 0,16 de las veces que no lo había, el modelo se confundió y determinó que si existía.

De manera análoga, el 91.87% de las veces que el verdadero estado era de fallo, el modelo lo predijo correctamente. Sin embargo, en un 8,13% de estas veces, no fue capaz de predecirlo y, por lo tanto, no alertó sobre ello.

Toma de decisiones de clasificación

En función de cuál sea la aplicación de nuestro modelo, puede ser más o menos crítico priorizar un tipo de fallo respecto a otro.

En el caso que estamos estudiando del aerogenerador, puede darse el caso de que se considere prioritario identificar todos los fallos, incluso a expensas de falsos positivos. Es decir, a expensas de veces en las que se cree que hay algún error, pero sin embargo el funcionamiento es correcto. En este caso, se priorizaría evitar fallos, aunque eso implicase un mayor gasto de mantenimiento derivado por hacer revisiones en ocasiones en las que no es necesario.

En otras ocasiones puede ocurrir que el coste de mantenimiento sea el factor prioritario y que por el contrario se consideren aceptables algunas averías fruto de no haber podido identificarlas. Esto podría ocurrir si, por ejemplo, se están monitorizando activos como aerogeneradores en zonas muy remotas donde movilizar al personal para hacer una revisión innecesaria es resulta muy costoso a nivel de recursos.

Esta decisión suele depender de un análisis de riesgo/beneficio y una evaluación de los costes tanto a nivel personal como económico (no es lo mismo la avería en un aerogenerador que en un avión). Una forma habitual de ajustar el comportamiento de un modelo es priorizar una métrica, ya sea el recall o la precisión. De esta forma, podemos priorizar un determinado comportamiento.

1.2 Optimización del umbral de clasificación

El umbral de clasificación es un parámetro importante a la hora de construir y evaluar modelos de clasificación. Puede influir significativamente en el rendimiento del modelo y en las decisiones tomadas a partir de sus predicciones.

Por defecto, una predicción positiva se clasifica cuando se alcanza un umbral superior al 50% por ciento.

Sin embargo, en función del problema específico y del equilibrio deseado entre precisión y recuperación, es posible que desee elegir un umbral diferente.

Vamos a repasar lo que era la precisión y el recall

- La **precisión** mide la relación entre los verdaderos positivos (TP) y todos los positivos predichos (TP + falsos positivos (FP)). Refleja la frecuencia con la que el modelo acierta cuando predice la clase positiva.
- El **recall** mide la relación entre los verdaderos positivos (TP) y todos los positivos reales (TP + falsos negativos (FN)). Refleja la proporción de casos positivos que el modelo puede detectar en el conjunto de datos.

Cada métrica tiene sus limitaciones. La precisión da prioridad a la "exactitud", pero puede no tener en cuenta el coste de los casos positivos omitidos. La recuperación hace hincapié en la "integridad", pero puede dar lugar a que se marquen falsamente algunos casos. Ambos tipos de errores pueden resultar caros, dependiendo del caso de uso específico. Por ello, se puede aplicar una de las siguientes circunstancias:

- Priorizar la Precisión (disminuir los Falsos Positivos): Si decides aumentar el umbral, el modelo requerirá una mayor "confianza" antes de predecir una instancia como positiva, lo que debería aumentar la precisión (menos falsos positivos), pero también puede disminuir el recall (más falsos negativos). Esto es útil si las consecuencias de un falso positivo son altas.
- Priorizar el Recall (disminuir los Falsos Negativos): Por otro lado, si reduces el umbral, el modelo predecirá más fácilmente una instancia como positiva, lo que debería aumentar el recall (menos falsos negativos) a costa de disminuir la precisión (más falsos positivos). Esto sería adecuado en situaciones en las que los falsos negativos tienen consecuencias graves.

Para obtener un modelo más equilibrado, o uno en el que tengamos una distribución más conveniente de verdaderos positivos y verdaderos negativos, podemos ajustar este umbral a un valor diferente, asegurándonos así de que estamos interpretando las probabilidades de acuerdo con nuestra estrategia de toma de decisiones.

En cualquier caso, es importante tener en cuenta que modificar demasiado el umbral provocará un ajuste excesivo.

Para realizar esta optimización, en primer lugar, vamos a ver el efecto de modificar el valor del umbral de clasificador.

9. Definimos manualmente un valor de umbral entre el 0 y el 1. A menor sea el valor, con menor será la probabilidad necesaria para considerar una predicción como positiva. A mayor valor, más exigente será el modelo antes de dar esa salida en forma de predicción.

   [Ver código en colab]

   Se puede observar como cuando bajamos el umbral, aumenta la precisión. Por el contrario, al subirlo, aumenta el recall. De esta manera, umbrales más bajos favorecen la detección de la clase positiva asumiendo errores de falsos positivos y umbrales más altos aseguran más la certeza de las predicciones positivas a costa de falsos negativos.

El balance entre precisión y recall puede representarse dráficamente a través de la **curva precisión recall**, que muestra los valores de ambas métricas obtenidas de un modelo de clasificación a varios umbrales. Por lo tanto, permite entender cómo los cambios en el umbral de clasificación afectan la capacidad del modelo.

10. Representamos la curva de Precision-Recall

```
plt.figure(figsize=(10, 8))
for YVar in YVars:
    y_true = dataframe_outcomes2[YVar]
y_scores = dataframe_prob_predictions2[YVar]
precision, recall, thresholds = precision_recall_curve(y_true,
y_scores)

    auc_score = auc(recall, precision)
```

```
        plt.plot(recall, precision, label=f'{YVar} (AUC={auc_score:.2f})')

    plt.xlabel('Recall') plt.ylabel('Precision')
    plt.title('Precision-Recall Curve by Type of Failure')
    plt.legend(loc='best') plt.grid(True) plt.show()
```

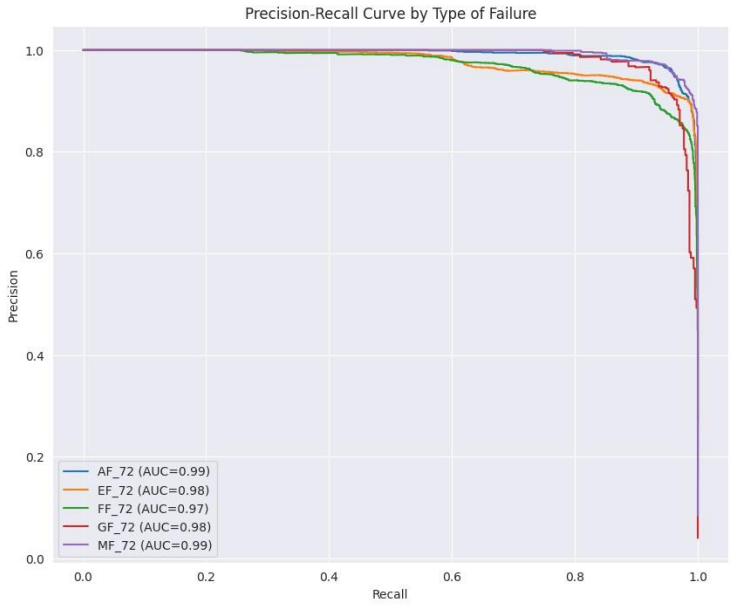


Figura 4: Curva Precision-Recall

Se puede observar que el gráfico tiene una tendencia descendente. Es decir, a medida de aumenta el recall, disminuye la precisión y viceversa. Los mejores resultados se corresponden con aquellos que están más cerca del vertice superior derecha. Ahí es donde se optimizan tanto los valores de precisión como de equilibrio. Por ello, a mayor area por debajo de la curva, mejor rendimiento general del modelo.

Ahora vamos a centrarnos en buscar el valor de umbral que maximiza nuestro objetivo.

Cuando lo que se busca es equilibrar el rendimiento del clasificador, normalmente suele buscar maximizar la métrica de F1.

- La **F1** es la media armónica de la precisión y el recall, y se utiliza para evaluar la precisión general de un modelo de clasificación binaria.

En nuestro caso, vamos a centrarnos en asegurarnos que somos capaces de detectar cuando ocurre un fallo con el mayor acierto posible. Esto implicaría aumentar el recall y evitar en la medida de lo posible los falsos negativas (FN). Por ello, podríamos fijar un umbral de decisión más bajo de manera que se priorice, incluso con probabilidades más bajas, que cualquier atisbo de fallo sea detectado.

Es posible que esto haga que el modelo tenga una precisión baja. De hecho, si sustituimos en el ejemplo anterior el umbral por 0, vemos que alcanzamos recalls perfectos. Sin embargo, esto no mejora nuestro modelo ya que la precisión se derrumba. Por ello, vamos a considerar para esta decisión la media de la F1 y el Recall, de manera que nos aseguremos de que el rendimiento del modelo siga siendo bueno y mejore el recall a expensas de perder algo de precisión.

11. Vamos a calcular el umbral que maximiza la media entre el F1 y el Recall. Es decir, aquel que consigue los mejores resultados ponderando ambas métricas. El siguiete código itera por todos los valores de umbral entre 0 y 1 en pasos de 0.1 y selecciona para cada modelo aquel en el que se obtienen los mejores resultados

    [Ver código en colab]

```
Para el tipo de fallo 'AF_72':
 - Con umbral por defecto: Precisión = 0.9893, Recall = 0.8018, F1-Score = 0.8858
 - Con umbral adaptado: Umbral óptimo = 0.3, Precisión = 0.9118, Recall = 0.9821, F1-Score = 0.9456

Para el tipo de fallo 'EF_72':
 - Con umbral por defecto: Precisión = 0.9333, Recall = 0.9237, F1-Score = 0.9285
 - Con umbral adaptado: Umbral óptimo = 0.33, Precisión = 0.8970, Recall = 0.9875, F1-Score = 0.9401

Para el tipo de fallo 'FF_72':
 - Con umbral por defecto: Precisión = 0.9470, Recall = 0.7761, F1-Score = 0.8531
 - Con umbral adaptado: Umbral óptimo = 0.24, Precisión = 0.8332, Recall = 0.9859, F1-Score = 0.9031

Para el tipo de fallo 'GF_72':
 - Con umbral por defecto: Precisión = 0.9601, Recall = 0.9233, F1-Score = 0.9413
 - Con umbral adaptado: Umbral óptimo = 0.36, Precisión = 0.8973, Recall = 0.9661, F1-Score = 0.9304

Para el tipo de fallo 'MF_72':
 - Con umbral por defecto: Precisión = 0.9717, Recall = 0.9459, F1-Score = 0.9586
 - Con umbral adaptado: Umbral óptimo = 0.31, Precisión = 0.9104, Recall = 0.9915, F1-Score = 0.9492

       default_precision  default_recall  default_fscore  optimal_threshold  \
AF_72           0.989339        0.801843        0.885778               0.30
EF_72           0.933333        0.923680        0.928482               0.33
FF_72           0.947030        0.776061        0.853063               0.24
GF_72           0.960094        0.923251        0.941312               0.36
MF_72           0.971678        0.945917        0.958624               0.31

       optimal_precision  optimal_recall  optimal_fscore
AF_72           0.911765        0.982143        0.945646
EF_72           0.896987        0.987518        0.940078
FF_72           0.833241        0.985860        0.903148
GF_72           0.897275        0.966140        0.930435
MF_72           0.910419        0.991516        0.949239
```

    De acuerdo con los resultados, podemos ver como umbrales en torno al 0,2-0,3 consiguen, en todos los modelos mejorar los valores de recall. Como contrapartida, esta ganancia se traslada en una reducción en la precisión, aunque el Fscore se mantiene de manera bastante similar el clasificador con el umbral por defecto.

De esta forma, habríamos encontrado un valor de umbral que nos permite tener un modelo más capaz de tomar las decisiones en base a nuestro criterio de prioridad.

2. Ajuste del modelo para detección multiclase

Ahora, en esta segunda parte, vamos a dejar a un lado el trabajo realizado para la creación de ventanas y selección de características.

Por ello, vamos a volver a los datos originales y combinar los datos del scada con los de los fallos.

Con ellos, vamos a entrenar un modelo multiclase que sea capaz no solo de determinar la existencia o no de un tipo de fallo (clasificación binaria) si no el fallo en sí.

12. Comenzamos por tanto importando esos archivos originales del dataset y combinándolos

```
scada_dataset = '/content/gdrive/MyDrive/00_ADI/00_COLAB/scada_data.csv' scada_df = pd.read_csv(scada_dataset)
scada_df['DateTime'] = pd.to_datetime(scada_df['DateTime'])

fault_dataset = '/content/gdrive/MyDrive/00_ADI/00_COLAB/fault_data.csv' fault_df = pd.read_csv(fault_dataset) fault_df['DateTime'] = pd.to_datetime(scada_df['DateTime'])

df_combine = scada_df.merge(fault_df, on='Time', how='outer')
```

13. Después nos aseguramos de limpiar el conjunto de datos de manera que, donde no haya valores (recordad que los datos se recolectaron en fechas diferentes), en vez de un NaN se aplique un valor de "NF" o No Fallo (Non Failure). Como vimos que se trataba de un dataset muy desbalanceado, vamos nuevamente a contar sus valores

```
df_combine['Fault'] = df_combine['Fault'].replace(np.nan, 'NF')
df_combine.Fault.value_counts()
```

El número de instancias de tipo NF en el dataset original es excesivamente superior a las de los fallos.

14. Por ello, vamos a reducir el número de instancias de No fallo a 300 para hacer una "undersampling" y balancear las clases de datos. Una vez reducidas, seleccionamos aquellas instancias que se correspondan a fallos y concatenamos ambos nuevos dataframes. Representaremos la distribución final de fallos.

```
df_nf = df_combine[df_combine.Fault=='NF'].sample(300, random_state=42)
df_f = df_combine[df_combine.Fault!='NF']
```

```
df_combine = pd.concat((df_nf, df_f),
axis=0).reset_index(drop=True)
```

Aunque en este ejercicio no vamos a realizar selección de características, sí que debemos eliminar aquellas que resultan irrelevantes para la clasificación.

15. Por ello, eliminamos las columnas de fecha, hora y error ya que únicamente estamos evaluando instancias correspondientes a instantes específicos.

```
train_df = df_combine.drop(columns=['DateTime_x', 'Time',
'Error', 'DateTime_y'])
train_df.Fault.value_counts().plot.pie(title='Fault Modes')
```

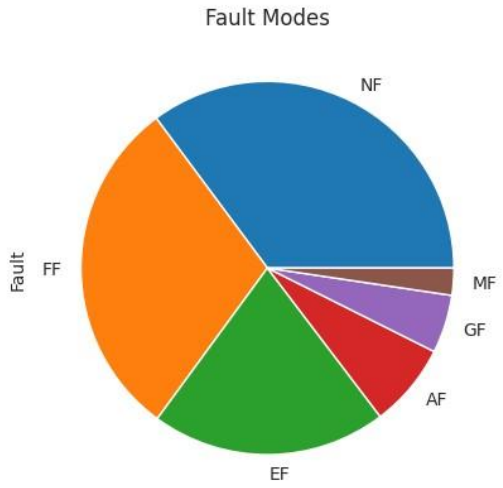


Figura 5: Distribución de las clases después del proceso de undersampling

A continuación, vamos a construir un modelo predictivo para clasificar los modos de fallo del aerogenerador basándonos en la información o estado de los componentes del aerogenerador. A diferencia de hasta ahora, en esta ocasión se trata de una tarea de clasificación multiclase.

Dado que nuestros datos de entrenamiento, incluso después del proceso de undersampling sigue muy desequilibrado para cada modo de fallo. Vamos a ver otra forma de balancear nuestras clases. En este caso, utilizaremos una técnica de oversampling llamada SMOTE (Synthetic Minority Oversampling Technique) para sobre muestrear las clases minoritarias. Esta técnica crea ejemplos sintéticos de la clase minoritaria en lugar de simplemente duplicar los existentes (como haría el sobre muestreo simple). Funciona seleccionando ejemplos que están cerca en el espacio de características (utilizando un algoritmo de KNN), dibujando una línea entre los ejemplos en el espacio de características y luego generando nuevos ejemplos a lo largo de esa línea.
Como algoritmo de clasificación utilizamos es LightGBM (Gradient Boosting Machine), un algoritmo se basa en el mismo principio de gradient boosting aplicándolo mediante un método más eficiente y optimizado.

A su vez, para evitar el sobreajuste, realizamos una validación cruzada (cross validation) estratificada K-Fold con 5 fold (tal y como vimos en los ejercicios del laboratorio pasado)

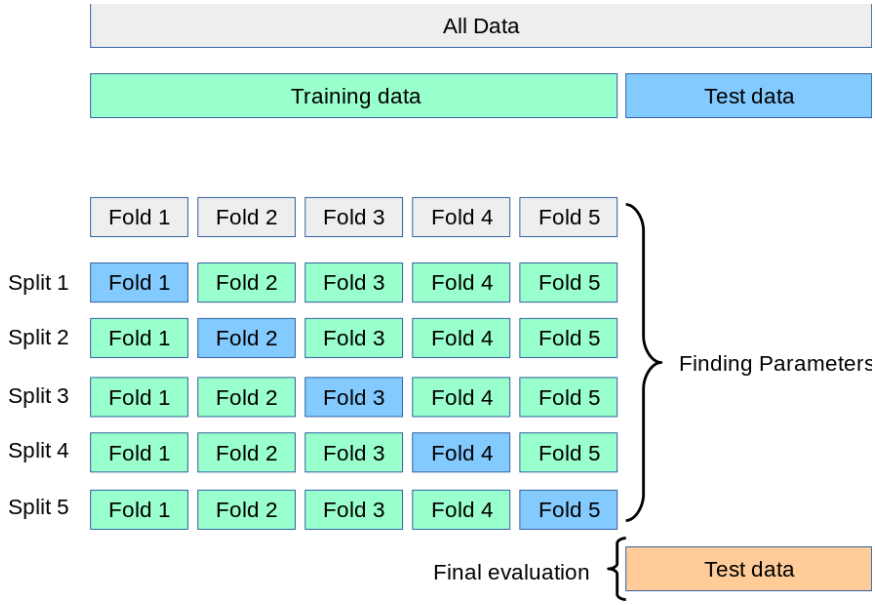


Figura 6: Proceso de cross-validation (https://scikitlearn.org/stable/modules/cross_validation.html)

En este proceso de crossvalidation, el conjunto de entrenamiento se dividirá en conjunto de entrenamiento y conjunto de validación en un número determinado de iteraciones (definidas por el número de folds). Por lo tanto, para asegurarse de que el conjunto de entrenamiento está equilibrado, el SMOTE debe hacerse únicamente con los datos de entrenamiento, ya que si no el resultado se vería afectado por data leackage.

Para ello, vamos a emplear el método de **pipeline**:
https://scikitlearn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

16. Extraemos los datos en datos de entrada (características) y de salida (output/clases) y dividimos los datos en entranamiento y est. Posteriormente creamos el pipeline con los pasos que queremos realizar durante el proceso de entrenamiento en el crossvalidation. Imprimimos los resultados del proceso de validación cruzada que se corresponden con la midia de los resultados de cada iteración.

```
X = train_df.iloc[:,:-1] y
= train_df.iloc[:,-1]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Make pipeline of SMOTE, scaling, and classifier
pipe = make_pipeline(SMOTE(), StandardScaler(),
LGBMClassifier(random_state=42))

# Define multiple scoring metrics scoring
= {
    'acc': 'accuracy',
    'prec_macro': 'precision_macro',
    'rec_macro': 'recall_macro',
```

```
    'f1_macro': 'f1_macro'
}

# Stratified K-Fold
stratkfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Return a dictionary of all scorings
cv_scores = cross_validate(pipe, X_train, y_train, cv=stratkfold,
scoring=scoring)

# Print scoring results from dictionary for
metric_name, metric_value in cv_scores.items():
    mean    =    np.mean(metric_value)              print(f'{metric_name}:
{np.round(metric_value, 4)}, Mean: {np.round(mean, 4)}')
```

Como era de esperar, obtenemos unos inferiores a los del laboratorio anterior. Esto se debe, en primer lugar, a que ya no estabamos tratando con modelos binarios donde se clasifica con 1 o con 0, si no que ahora tenemos un único modelo para 6 clases.

Por otro lado, el crear ventanas de 24/72/48h nos ayuda a reeducir la granularidad y a suavizar la variabilidad a corto plazo, lo que es útil para identificar tendencias más significativas. A su vez, esta agregación por ventanas aumenta la cantidad de datos de cada clase.

17. Comprobamos los resultados finales con el test set que habíamos reservado previamente. Esto es equivalente a usar train, test y validación

```
# Fit pipeline to train set pipe.fit(X_train,
y_train)

# Predict on test set y_pred
= pipe.predict(X_test)

# Classification report print(classification_report(y_test,
y_pred))
```

Los resultados que obtenemos están en línea con los de la fase de cross-validation.

Sin embargo, todavía podemos intentar mejorar el modelo.

Esto implica ajustar los parámetros para optimar los resultados para la métrica deseada. Para ello podemos emplear un enfoque de búsqueda en cuadrícula (Grid Search) para probar sistemáticamente combinaciones de parámetros y encontrar la configuración óptima de los hiperparametros del algoritmo LightGBM.
De esta forma, automáticamente se prueban todas las combinaciones posibles de hiperparámetros el conjunto de datos de entrenamiento y se evalúan mediante el proceso de cross-validation, priorizando aquella métrica que se considere de interés.

18. Realizamos el ajuste de parámetros y entrenamos un nuevo modelo con la combinación resultante, buscando optimizar nuevamente la métrica recall

```
# Define parameter search grid
param_grid = {'lgbmclassifier__n_estimators': [6, 8, 16, 24],
              'lgbmclassifier__num_leaves': [4, 6, 8],
              'lgbmclassifier__reg_alpha' : [1, 1.5],
              'lgbmclassifier__reg_lambda': [1, 1.5],
              'lgbmclassifier__boosting_type': ['gbdt'] #
Gradient Boosting Decision Tree
             }

# Grid search CV
grid = GridSearchCV(pipe, param_grid, verbose=1, cv=stratkfold,
n_jobs=-1, scoring = 'recall_macro')

# Fit grid on train set
grid.fit(X_train, y_train) #
Best model from tuning
print(grid.best_params_)

# Classification report y_pred =
grid.predict(X_test)
print(classification_report(y_test, y_pred))
```

Podemos ver que los resultados de recall han mejorado ligeramente después de este ajuste de hiperparámetros, indicando que el modelo es más efectivo en identificar correctamente todas las instancias positivas.